Caleta: An Evidence-Oriented Trust Layer for Reproducible Local Large Language Model Inference

Technical Preprint — Software Validation Manuscript

Joan-Marc Fisa
BiVelio Inc
jmfisa@bivelio.com

31 October 2026

Copyright © 2026 BiVelio Inc. This manuscript is licensed under Creative Commons Attribution 4.0 International (CC BY 4.0).

Corresponding author: Joan-Marc Fisa (jmfisa@bivelio.com).

Software reference implementation: Caleta v0.1.0, Apache-2.0.

Code and evidence: https://github.com/BiVelio/caleta.

Zenodo DOI: 10.5281/zenodo.23836099.

Status: non-peer-reviewed technical preprint.

Abstract

Local large language model (LLM) inference is often described as private, verified, or reproducible without separating the technical claims behind those terms. This paper presents **Caleta**, an Apache-2.0 process supervisor and evidence layer around llama.cpp rather than a new inference engine. Caleta combines versioned manifests, exact-size and SHA-256 verification, controlled backend construction, a restricted OpenAI-compatible proxy, prompt-free execution records, benchmark reports, and a small conformance suite. Its contribution is the composition of these mechanisms into explicit contracts that distinguish integrity from authenticated provenance and privacy controls from operating-system isolation. We evaluate the implementation through static checks, fifteen unit tests, deterministic end-to-end tests on Ubuntu, macOS, and Windows, and a pinned real-model experiment on Windows CPU using Qwen2.5-0.5B-Instruct Q4_K_M and llama.cpp b11981. The real experiment validates artifact verification, startup, model listing, non-streaming and server-sent-event chat, and conformance. Three independent benchmark invocations produced fifteen successful warm requests with no failures. In this single configuration, median time to first token was 18.066 ms and median decode throughput was 54.394 tokens/s. These descriptive measurements do not establish cross-engine superiority, output quality, energy efficiency, or hardware-general performance. Raw JSON, commands, and environment metadata are published. Remaining challenges include signed manifests, time-of-check/time-of-use replacement, operating-system containment, proxy-overhead measurement, and broader experimental matrices.

Keywords: local inference; large language models; reproducibility; software supply chain; privacy engineering; benchmarking; provenance; llama.cpp.

1 Introduction

The deployment of LLMs on local machines is motivated by latency, cost, data-control, availability, and operational-sovereignty requirements. Open-source runtimes such as llama.cpp, MLX-based systems, Ollama, Uzu, and BaseRT address important parts of this problem, including hardware coverage, model packaging, ease of use, and optimized execution [1, 2, 3, 4, 5]. However, the ability to execute a model locally does not by itself answer several operational questions: Which bytes were executed? Which backend and runtime options were active? Which interfaces were exposed? What was measured, and which runs failed? What evidence remains after execution?

These questions are often conflated under broad claims of privacy or verification. A cryptographic digest can establish correspondence between an artifact and an expected value, but not the identity of the publisher who supplied that value. An offline backend flag can request network inactivity, but it is not equivalent to an operating-system firewall. A local HTTP listener can avoid wide-area exposure while remaining reachable by other local processes. Similarly, a throughput number without artifact identity, generation parameters, cold/warm separation, and failed-run accounting is insufficient to reproduce an experiment.

Caleta addresses this narrower but consequential layer around the inference engine. It deliberately does not implement tensor kernels, a model format, or a new serving API. Instead, it wraps llama.cpp's `llama-server` with a small trust boundary and emits versioned evidence. This choice preserves GGUF interoperability [9] and allows the measurement and conformance contracts to be reused independently of the supervisor.

The manuscript makes three deliberate evidentiary restrictions. First, it reports only tests actually executed on the reference implementation. Second, it separates deterministic mock-based orchestration tests from a real-model experiment on one Windows CPU host. Third, it treats authenticated provenance, OS-level containment, cross-engine comparison, and broader real-model matrices as future work rather than implied properties of the current system.

2 Research Questions and Contributions

We organize the work around three research questions:

RQ1:
How can a local inference launcher make artifact integrity and runtime configuration explicit without replacing the inference engine?

RQ2:
Which operational and benchmarking evidence can be emitted without persisting prompts and responses by default?

RQ3:
How can deterministic tests and a real-model experiment complement each other while keeping their evidentiary scopes distinct?

The paper offers four contributions:

1. A composable architecture that combines artifact verification, backend supervision, restricted proxying, execution records, benchmarking, and conformance under explicit security boundaries.
2. Versioned manifest and report contracts that separate declared provenance, observed integrity, runtime configuration, and measurement limitations.
3. Benchmark definitions that preserve raw failures, distinguish cold startup from warm requests, and avoid converting unavailable measurements into zero-valued results.
4. A two-layer evaluation comprising cross-platform deterministic tests and a pinned real-

GGUF experiment with raw reports, commands, environment metadata, and a threat-to-validity analysis.

The contribution is therefore not a universal performance improvement or a formal proof of privacy. It is a reference design for bounded, inspectable claims in local inference systems.

3 Background and Related Work

3.1 Local inference runtimes

llama.cpp provides broad GGUF and hardware support and includes an HTTP server with offline, authentication, and cross-origin resource sharing (CORS) options [1]. Caleta does not assume that these upstream controls are absent. Its purpose is to construct a consistent subset of configuration, remove selected inherited overrides, restrict externally reachable routes, and produce execution and measurement evidence around that configuration.

Ollama emphasizes installation and model-distribution ergonomics [3]; MLX LM integrates local inference with Apple Silicon and the Hugging Face ecosystem [2]; Uzu implements a Rust inference engine with platform-specific optimization and a broader application ecosystem [4]. BaseRT demonstrates the importance of native Metal optimization and phase-specific performance evaluation on Apple Silicon [5]. These projects primarily address execution, optimization, packaging, or developer experience. Caleta is complementary: its schemas and conformance checks could be implemented by another runtime without adopting its process supervisor.

The BaseRT preprint also illustrates why performance claims must be scoped by hardware, model family, quantization, and inference phase. Its results concern the configurations evaluated by its authors. They are neither replicated in this study nor used as evidence for Caleta's performance.

3.2 Integrity, provenance, and attestation

Secure software distribution requires more than file hashing. SHA-256 provides a deterministic digest used here to compare artifact contents against a manifest [6]. The security of that comparison depends on how the expected digest was obtained. Supply-chain frameworks such as SLSA distinguish provenance and build properties from the simplified notion that a checksum alone establishes trust [7].

Caleta v0.1.0 implements integrity comparison and provenance declaration. It does not implement signed manifests, transparency logs, remote attestation, or independently signed execution records. We reserve *authenticated provenance* for mechanisms that establish the identity of the party supplying an artifact or manifest through a trusted channel.

3.3 Reproducibility and transparency in AI

Reproducibility research distinguishes the artifacts, computational environment, methodology, and analysis needed to obtain similar results [11, 12]. Model cards and datasheets provide structured documentation for trained models and datasets, including their intended uses and limitations [13, 14]. Caleta operates at a complementary layer: its manifest and execution reports describe what was run and how, not how the model was trained, its full evaluation profile, or its suitability for a task. A Caleta manifest therefore should accompany, rather than replace, model and dataset documentation.

3.4 Reproducible performance evaluation

Local inference comprises at least model initialization, prompt processing, and autoregressive decoding. Combining these stages into one number obscures different system behaviors. Meaningful comparisons must hold artifact identity, quantization semantics, prompts, templates, decoding parameters, backend revisions, hardware, and power conditions as constant as practical [12]. They must also distinguish cold starts from warm requests and describe how tokens and timing events are counted.

Caleta operationalizes part of this methodology in JSON. It does not measure output quality, energy, thermal throttling, GPU memory, or pure kernel time. These omissions are represented as limitations rather than estimated values.

4 Design Principles

The architecture follows five principles:

Composition over replacement. The reference backend remains llama.cpp. Caleta does not duplicate its kernels, model conversion, or hardware backends.

Verification before execution. `serve` and `bench` refuse to start after a missing artifact, size mismatch, digest mismatch, or read failure.

Least exposed interface by default. Both the public proxy and backend default to loopback. The backend is never intentionally exposed on a public listener. Only health, model listing, and chat completions are proxied.

Evidence with bounded meaning. Every contract names what it records and what it does not establish. A verification report is labeled `integrity_only`; execution records are unsigned operational evidence, not attestations.

Reproducibility without prompt retention by default. Configuration and measurements are retained, while request and response bodies are not logged. Benchmarks store a prompt digest and length unless the operator explicitly requests prompt inclusion.

5 System Architecture

5.1 Components and data flow

Figure 1 shows the principal boundary. A client communicates with the Caleta proxy. The proxy replaces external authorization with a random per-launch internal credential and forwards only supported requests to llama-server. The backend binds to IPv4 loopback and receives a verified local GGUF path.

The implementation separates manifest parsing, verification, backend lifecycle, HTTP serving, execution records, diagnostics, benchmarking, and conformance. This decomposition permits unit testing without a model and keeps the command-line interface from becoming the sole integration point.

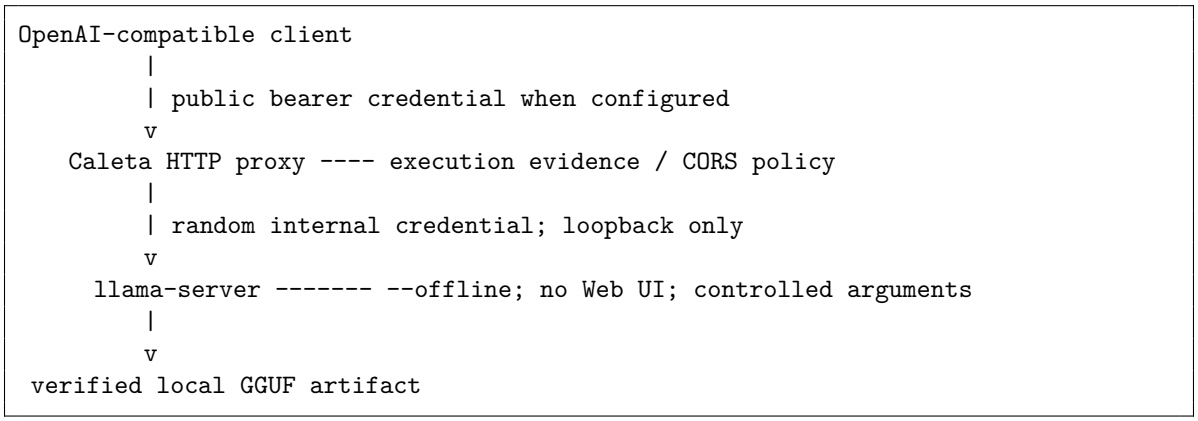


Figure 1: Caleta process and evidence boundary. The diagram does not imply OS-level sandboxing.

5.2 Manifest and artifact verification

A manifest declares a schema identifier, model identifier, license, source, artifact path, expected digest and size, backend kind, and runtime parameters. Relative artifact and explicit backend paths resolve against the manifest directory rather than the caller's working directory. The source and license are declarations; they are not inferred from file contents.

For artifact bytes a, declared digest h, and declared byte length b, execution proceeds only if

$$|a| = b \quad \wedge \quad \mathrm{SHA256}(a) = h. \tag{1}$$

Verification streams data in 1 MiB blocks, so artifact size does not determine verifier memory usage. When a backend digest is pinned, an explicit backend path is required to avoid treating ambiguous `PATH` resolution as precise provenance.

This check does not establish that the manifest is truthful, that the model is safe to parse, or that the file cannot be replaced between verification and backend loading. Signed manifests and immutable or otherwise protected storage are therefore separate requirements.

5.3 Backend supervision

Before launch, Caleta removes inherited `LLAMA_ARG_*` and `LLAMA_API_KEY` variables that could alter the declared boundary. It generates a fresh 32-byte random internal credential, passes it through the backend environment, and constructs arguments for model, loopback host, ephemeral port, alias, offline mode, disabled Web UI, context size, GPU layers, and optional thread count.

The manifest validator rejects additional arguments that would replace the model or listener, alter credentials, or enable agent tools. This is a reviewed policy for known options, not a universal proof against every future upstream flag. Readiness requires an authenticated health response within a timeout. Premature child exit and readiness timeout are errors. Shutdown terminates and waits for the child, with kill-on-drop as an additional process-lifetime safeguard.

Two residual races remain: another local process may bind a reserved ephemeral port before the child uses it, and an artifact may be replaced after verification. Hostile-local-user deployments require stronger OS-specific mechanisms.

5.4 Restricted HTTP proxy

The public server exposes only `/health`, `/v1/models`, and `/v1/chat/completions`, a deliberately small subset of the OpenAI-compatible API [8]. Non-loopback listeners require a public bearer key of at least sixteen characters, although high-entropy random secrets are recommended. Credential comparison is constant-time. Request bodies are limited to 16 MiB; responses, including server-sent events (SSE), are streamed rather than collected.

The proxy removes hop-by-hop headers and all upstream CORS response headers. Only Caleta's explicit origin allowlist can produce CORS headers. Wildcard origins are rejected. The server provides plain HTTP; remote deployments require a TLS reverse proxy, SSH tunnel, or an appropriately controlled private network. CORS is not an authentication mechanism and does not protect non-browser clients.

6 Operational Evidence and Measurement Contracts

6.1 Execution records

Optional execution records use the schema `caleta.dev/execution/v1`. They capture version, model identity, paths, artifact and backend digests, listener configuration, loopback/offline flags, CORS policy, arguments, and limitations. Prompt and response bodies are excluded. The record is generated by the same process whose behavior it describes and is unsigned; it supports inspection and reproduction but does not prove execution to a third party.

6.2 Benchmark methodology

A benchmark verifies the artifact, starts one backend, measures cold startup to authenticated readiness, executes warm-up requests, and then measures warm requests in that process. Failed measured requests remain in the report and are excluded only from aggregates of successful measurements. Warm-up failures are logged but are not retained as raw per-run records. A report is written even when all measured requests fail.

For a request sent at t_0, the first content-bearing SSE event arrives at t_1, and the last such event arrives at t_n. Time to first token is defined operationally as

$$\mathrm{TTFT} = t_1 - t_0. \tag{2}$$

It includes local HTTP and queueing overhead and ignores empty role-only events. If the backend reports c completion tokens and $c \geq 2$, decode throughput is

$$\mathrm{decode_tps} = \frac{c - 1}{t_n - t_1}. \tag{3}$$

Observed prefill throughput is reported prompt tokens divided by TTFT; it is not a pure kernel measurement. Repeating an identical prompt within one backend process may also reuse cached prompt state, so this value must not be interpreted as cold prefill performance. Inter-event latency is computed over arrivals of content-bearing SSE events. Because backend chunking need not map one event to one tokenizer token, this metric is not claimed to be exact per-token latency in all backends.

The schema `caleta.dev/benchmark/v1` stores raw runs, token usage, estimated-token flags, cold startup, configuration, environment, failures, and limitations. On Linux, main-process resident memory is sampled approximately every 50 ms through `/proc/PID/status`; this excludes GPU memory and child processes and can miss short peaks. Missing metrics remain null.

6.3 Conformance

The conformance command checks a deliberately small OpenAI-compatible contract: model listing returns a data array, non-streaming chat contains message content in the expected structure, and streaming chat uses `text/event-stream`, emits deltas, and terminates with `data: [DONE]`. The schema is `caleta.dev/conformance/v1`. Passing does not certify tool calls, structured output, multimodality, cancellation, backpressure, fragmented SSE handling, or complete OpenAI compatibility.

7 Implementation and Reproducibility

The reference implementation is written in Rust and uses Tokio, Axum, Reqwest, Serde, and SHA-2. The package declares Rust 1.88 as its minimum version and Apache-2.0 as its software license. Model artifacts and llama.cpp retain their respective licenses. Caleta does not bundle a model or llama.cpp binary.

A minimal local workflow is:

```
cargo build --release --locked
caleta doctor --backend /path/to/llama-server
caleta verify model.toml
caleta serve model.toml --record-out execution.json
caleta bench model.toml --output benchmark.json \
  --runs 5 --warmup 1 --max-tokens 64 --seed 42
caleta conformance --url http://127.0.0.1:8080 \
  --output conformance.json
```

The canonical public artifact location is:

https://github.com/BiVelio/caleta

The repository contains the source, lockfile, manifests, and deterministic mock. Future experimental publications must also include raw reports and exact command transcripts.

8 Evaluation

8.1 Experimental scope

The evaluation has two layers. The first uses a deterministic Python backend that accepts the subset of llama-server arguments used by Caleta, requires the internal credential, and returns fixed protocol responses. Its 45-byte artifact is not a valid GGUF model. This layer tests orchestration and protocol behavior without claiming inference. Local validation used Windows with rustc 1.94.1, Cargo 1.94.1, and Python 3.14.0; hosted CI subsequently covered Ubuntu, macOS, and Windows.

The second layer executes a real Qwen2.5-0.5B-Instruct Q4_K_M GGUF through pinned llama.cpp binaries on one Windows CPU host. It tests artifact verification, real model startup, API behavior, conformance, and descriptive inference timing. It does not evaluate output quality, safety, energy, or cross-engine performance.

8.2 Procedure

The validation sequence was:

```
cargo fmt --all -- --check
cargo clippy --locked --all-targets -- -D warnings
cargo test --locked
cargo build --locked
CALETA_BIN=target/debug/caleta python3 tests/e2e_mock.py
```

The end-to-end scenario constructs a temporary artifact and manifest, computes their digests, and executes verification. It starts the supervised service, checks unauthenticated rejection, performs authorized model listing and streaming chat, and runs the conformance command. It then executes one warm-up and two measured benchmark requests and inspects the resulting JSON. The Unix branch of the scenario validates graceful interruption and the resulting prompt-free execution record; that branch was not executed in the local Windows run. The Windows scenario uses forced process-tree shutdown for the mock and does not claim graceful-shutdown or execution-record coverage.

8.3 Results

All reported checks completed successfully after correcting platform-specific implementation and test issues: an unused non-Linux process identifier in the benchmark module rejected by strict Clippy, and Windows handling of TOML paths, executable mock launching, and mock-process shutdown in the test harness. Table 1 maps outcomes to the claims they support.

Check	Observed result	Supported interpretation
Formatting	Pass	Source conforms to Rust formatting.
Clippy, all targets	Pass with warnings denied	No emitted lint warnings under the configured toolchain.
Unit tests	15 passed, 0 failed	Tested manifest, digest, SSE, percentile, authentication, proxy, and exposure cases behave as expected.
Locked build	Pass	Package builds with the committed dependency resolution.
Verification E2E	Pass	Exact size/digest and pinned mock backend are accepted by the workflow.
Serve/authentication E2E	Pass	Supervised proxy starts; missing public credentials are rejected; authorized models and chat requests succeed.
SSE/conformance E2E	Pass	The tested stream and three conformance contracts pass against the deterministic backend.
Benchmark E2E	2 successful runs	Raw successful-run records and model digest are emitted; no prompt text is stored by default.
Mock-based inference claims	Not made	The fixture validates orchestration only; real-model evidence is reported separately below.

Table 1: Local software-validation outcomes and their bounded interpretation.

8.4 Hosted multi-platform confirmation

After the local validation, GitHub Actions run 41198343615 completed successfully at source revision `74183995aac0a87f61598ab755d0f72f6358aae7`. The Ubuntu, macOS, and Windows

test jobs passed, as did the separate formatting and Clippy job. The Ubuntu and macOS end-to-end runs included graceful-shutdown execution records; the Windows run used forced mock-process shutdown. The committed JSON and logs preserve all three runner results, including the 45-byte mock artifact and the 15 passing unit tests. This confirms the same mock-based contracts across the three hosted operating systems, but it does not add real-model evidence.

No tokens-per-second result from the mock is scientifically meaningful. Mock timing is intentionally excluded from the performance claims.

8.5 Real-model procedure

The real experiment used Qwen2.5-0.5B-Instruct Q4_K_M under Apache-2.0, pinned at Hugging Face revision `9217f5db79a2` [10]. The artifact has 491,400,032 bytes and this publisher-declared SHA-256:

```
74a4da8c9fdbcd15bd1f6d01d621410d31c6fc00986f5eb687824e7b93d7a9db
```

The backend was the Windows CPU asset from llama.cpp release `b11981`. Both the release archive and executable were hash-pinned. Caleta was built from revision `539ed3e32b09`. Full identifiers and digests are preserved in the public experiment artifacts.

The host was an MSI MS-7A78 running Windows 10 Home 10.0.19045 with an Intel i7-7700K, four physical cores, and 15.96 GiB RAM. Execution was CPU-only with four threads, a 2,048-token context, `-jinja`, seed 42, temperature zero, and a 64-token output limit. Power state, thermal state, and unrelated system load were not measured or controlled.

After successful `doctor` and `verify` checks, Caleta started the real backend on loopback. The experiment retained health and model-list responses, a non-streaming chat response, a streaming SSE session with usage and `[DONE]`, and the conformance report. The service and backend were stopped before benchmarking. Three independent benchmark invocations each performed one cold start, two warm-ups, and five measured requests with the same prompt.

8.6 Real-model results

Verification passed for the real model and backend. Model listing, non-streaming chat, and SSE streaming completed successfully, and all three conformance checks passed. All fifteen measured benchmark requests succeeded; no failures required exclusion. Table 2 reports descriptive aggregates computed from the three raw reports.

Metric	Observed value	Scope
Independent cold starts	3	One per benchmark invocation
Measured warm requests	15	Five per invocation; zero failures
Cold start, median	835.371 ms	Range 660.720–977.129 ms
TTFT, median	18.066 ms	Range 14.574–28.004 ms
Decode throughput, median	54.394 tokens/s	Range 49.028–68.138 tokens/s
Decode sample standard deviation	6.617 tokens/s	Descriptive; not a confidence interval

Table 2: Single-host real-model measurements. They are not cross-engine or hardware-general claims.

Repeated prompts within each backend process may benefit from caching. TTFT includes HTTP and queueing, and observed prefill values are not cold kernel measurements. The experiment does not estimate public-proxy overhead because Caleta's benchmark talks directly to its supervised backend. It also does not measure energy, memory, output quality, safety, or thermal

behavior. Retained chat responses provide protocol evidence, not an exact-instruction-following evaluation.

The manifest, commands, environment record, raw JSON reports, SSE session, and aggregate summary are publicly available in the real-GGUF experiment directory. No model or backend binary is redistributed there.

For RQ1, the real experiment confirms that the manifest, verification, supervision, and proxy operate with a pinned GGUF and llama.cpp build, complementing the deterministic tests. For RQ2, the experiment provides raw operational and benchmark evidence without private prompts; hosted Unix runs separately validate prompt-free execution records. For RQ3, the two layers demonstrate why protocol determinism and real-model validation are complementary rather than interchangeable evidence.

8.7 Artifact traceability

Every empirical claim in this evaluation is backed by a committed data file or a committed script. Table 1 is aggregated by tests/aggregate_validation.py from preserved runner outputs, while Table 2 is regenerated by aggregate_results.py –check from the three raw benchmark reports. The osaudit configuration links the paper to these results and instruments, and returns no findings at the evidence tag. This audit establishes mechanical traceability, not independent scientific correctness, external replication, or peer review.

9 Security and Privacy Analysis

Table 3 summarizes the implemented controls and residual risks. The central analytical requirement is to avoid converting a control in one layer into a guarantee about the whole system.

Threat or ambiguity	Current control	Residual risk or non-claim
Artifact substitution	Exact size and SHA-256 before serve/bench	Trusted digest distribution and post-check replacement remain unresolved.
Backend substitution	Optional explicit-path digest	Unpinned PATH resolution remains possible and is reported as such.
Inherited reconfiguration	Remove selected llama.cpp environment options	Other host variables, drivers, and future upstream options remain relevant.
Unauthorized network API use	Bearer authentication outside loopback	No integrated TLS; secret storage and transport are operator responsibilities.
Permissive upstream CORS	Strip upstream CORS; explicit local allowlist	CORS does not authenticate non-browser clients.
Backend network access	Loopback listener and offline request	No OS-enforced outbound-network denial.
Prompt retention by Caleta	No request/response logs; prompt digest by default	Clients, backend, OS swap, crash reports, and other processes are separate.
Misleading execution evidence	Versioned prompt-free JSON	Records are unsigned and are not remote attestations.

Table 3: Security controls and residual risks in Caleta v0.1.0.

A fully malicious host operating system, privileged local attacker, compromised Caleta binary,

or malicious native backend is outside the containment claimed by this version. Model parsing occurs inside llama.cpp, so a matching digest cannot make an untrusted GGUF safe to parse. Loopback also is not a complete security boundary against other local processes.

Accordingly, we define the supported privacy claim narrowly: Caleta adds no telemetry or prompt history, avoids logging request and response bodies, requests offline backend behavior, and limits backend network listening to loopback. It does not claim that the complete host cannot disclose data.

10 Threats to Validity

Construct validity. Integrity, declared provenance, authenticated provenance, privacy, and safety are distinct constructs. The implementation and reports use separate terms, but users could still overinterpret the word "verified." The `integrity_only` label and explicit limitations mitigate rather than eliminate this risk.

Internal validity. The mock backend does not reproduce GGUF parsing, model initialization, tokenization, or hardware behavior. The real experiment covers those mechanisms for one pinned artifact and backend revision, but cannot expose defects specific to other models, versions, or configurations. Repeated prompts may reuse cached state, and the Windows mock uses forced process-tree shutdown, so graceful-shutdown evidence remains platform-specific.

External validity. Hosted mock-based CI on three operating systems and one real-model Windows CPU experiment cannot establish correctness or performance across backend revisions, model families, hardware accelerators, or adversarial conditions. No macOS or Linux real-model run is claimed. The results should be generalized only through additional pinned experiments.

Reliability. The committed lockfile, exact source and backend revisions, immutable model revision, publisher digests, commands, environment record, raw JSON, and versioned schemas support repetition of the reported experiment. Absolute local paths in diagnostic evidence and uncontrolled host load can affect exact repetition; timing values are not guaranteed to reproduce.

Performance validity. Three cold starts and fifteen warm requests provide descriptive evidence for one host, not a stable performance distribution or comparative ranking. The study cannot estimate public-proxy overhead, energy, thermal effects, output quality, or cross-hardware performance. The sample standard deviation is reported without treating it as a confidence interval, and no claims are made about prefill kernels, memory, or quality.

11 Research Agenda

The next stage must add evidence rather than broaden marketing language. We identify four ordered directions.

11.1 Real-model experimental matrix

A preregistered matrix should pin at least one redistributable GGUF artifact and llama.cpp revision per claimed operating system. Experiments should record model source and digest,

quantization, context, prompt template, seed, temperature, hardware, power state, backend options, and Caleta revision. Direct-backend and proxied measurements should estimate overhead with repeated independent runs and uncertainty intervals. Raw successful and failed runs should be published.

11.2 Authenticated provenance and release evidence

Signed manifests, protected update channels, release attestations, and independently reproducible builds should be evaluated as separate mechanisms. Acceptance tests must demonstrate signature verification and failure under modification. A software bill of materials may complement these controls but cannot replace them.

11.3 Operating-system containment

Network and filesystem restrictions require platform-specific enforcement and adversarial tests. A backend flag must not be reported as a sandbox. The acceptance criterion is a demonstrated blocked outbound connection and denied unauthorized resource access under the stated threat model.

11.4 Protocol and measurement robustness

Conformance should expand to fragmented SSE, cancellation, concurrent requests, backpressure, structured outputs, and tool calls only when each contract is implemented and tested. Measurement research should add calibrated energy, GPU and process-tree memory, thermal state, and quantization-quality evaluation without conflating their definitions across platforms.

12 Conclusion

Caleta explores a complementary layer in the local-LLM stack: making artifact identity, runtime configuration, exposed interfaces, and measurement methodology explicit and machine-readable. Its design avoids duplicating optimized runtimes and instead composes integrity checks, supervision, restricted proxying, prompt-free operational evidence, benchmarking, and conformance around llama.cpp.

Hosted deterministic tests on three operating systems and a pinned real-GGUF experiment on Windows CPU validate the tested behavior, including real startup, chat, streaming, conformance, and benchmarking. The real measurements are deliberately scoped to one configuration and are not a comparative performance claim. This distinction is central to the contribution: trustworthy local inference requires claims whose scope is visible in both documentation and artifacts. Caleta provides an open implementation and reproducible evidence for that principle, with future work needed on broader matrices, authenticated provenance, containment, and measurement.

Author and Intellectual Property

Author and corresponding author: Joan-Marc Fisa, BiVelio Inc, `jmfisa@bivelio.com`.
Manuscript copyright and license: Copyright © 2026 BiVelio Inc. Distributed under CC BY 4.0; attribution must identify Joan-Marc Fisa as author and BiVelio Inc as copyright holder.
Reference software license: Apache License 2.0. Third-party software and model artifacts

retain their own licenses. The software license does not grant trademark rights in Caleta or BiVelio.

Data and Code Availability

The reference implementation, test harness, documentation, and release runbook are available at https://github.com/BiVelio/caleta. The real-model manifest, commands, environment record, raw reports, and aggregate summary are available in the experiment artifacts directory. This manuscript version is assigned DOI 10.5281/zenodo.23836099; the earlier version remains archived at 10.5281/zenodo.23763881.

References

[1] ggml-org. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp, accessed 28 October 2026.

[2] Apple Machine Learning Research. MLX LM: LLM inference with MLX. https://github.com/ml-explore/mlx-lm, accessed 28 October 2026.

[3] Ollama contributors. Ollama: local model execution and distribution. https://github.com/ollama/ollama, accessed 28 October 2026.

[4] Mirai contributors. Uzu: a Rust large language model inference engine. https://github.com/trymirai/uzu, accessed 28 October 2026.

[5] Prabod Rathnayaka, Fabian Waschkowski, and Lukas Wesemann. BaseRT: Best-in-Class LLM Inference on Apple Silicon via Native Metal. arXiv:2607.00501 [cs.CL], 2026. https://arxiv.org/abs/2607.00501.

[6] National Institute of Standards and Technology. FIPS PUB 180-4: Secure Hash Standard (SHS). 2015. https://csrc.nist.gov/pubs/fips/180-4/upd1/final.

[7] OpenSSF and SLSA contributors. Supply-chain Levels for Software Artifacts (SLSA) Specification. https://slsa.dev/spec/, accessed 28 October 2026.

[8] OpenAI. API Reference: Models and Chat Completions. https://platform.openai.com/docs/api-reference, accessed 28 October 2026.

[9] ggml-org. GGUF model format specification. https://github.com/ggml-org/ggml/blob/master/docs/gguf.md, accessed 28 October 2026.

[10] Qwen. Qwen2.5-0.5B-Instruct-GGUF: model repository and license. https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct-GGUF, accessed 29 October 2026.

[11] Odd Erik Gundersen and Sigbjorn Kjensmo. State of the Art: Reproducibility in Artificial Intelligence. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1):1644–1651, 2018. https://doi.org/10.1609/aaai.v32i1.11503.

[12] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d'Alche-Buc, Emily Fox, and Hugo Larochelle. Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program). *Journal of Machine Learning Research*, 22(164):1–20, 2021. https://jmlr.csail.mit.edu/papers/v22/20-303.html.

[13] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model Cards for Model Reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019. https://doi.org/10.1145/3287560.3287596.

[14] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daume III, and Kate Crawford. Datasheets for Datasets. *Communications of the ACM*, 64(12):86–92, 2021. https://doi.org/10.1145/3458723.